

第一章

文件I/O

C语言函数分类

- ◆ C语言函数按照提供源分类
 - ◆ C语言标准函数
 - ◆ 操作系统提供的函数
 - ◆ 自定义函数
 - ◆ 第三方函数

文件基本概念

◆Linux常见的文件类型：

- ◆普通文件（一般的二进制文件，mp3,mp4,txt文本等）
- ◆目录文件
- ◆设备文件（硬盘驱动器，磁带驱动器，网络当中其中的一个节点）
- ◆管道文件（用于进程之间的通信）
- ◆套接字（类似于管道，管道只能在本地通信，套接字可以在网络中通信）
- ◆链接文件（这个文件呢包含了另外一个文件的路径，相当于windows下面的一个快捷方式）

普通文件

- ◆ 普通文件 (regular file) :

- ◆ 文本文件
- ◆ 二进制文件

```
[alex@alex~]$ls -l minicom.log
```

```
-rw-rw-r-- 1 alex alex 2903829 Apr 23 22:41 minicom.log
```

目录文件

- ◆ 目录文件 (directory) , 主要目的是用于管理和组织系统中的大量文件

```
[alex@alex~/spec]$ls -l  
  
total 8784  
  
- rw-r--r-- 1 alex alex 8970378 Dec 13 12:56 2800000 1.pdf  
  
drwxrwxr-x 2 alex alex 4096 Nov 23 09:13 uart
```

设备文件

- ◆设备文件，Linux操作系统把每一个I/O设备都看成一个文件，与普通文件一样处理，这样可以使文件与设备的操作尽可能统一，对I/O设备的使用和一般文件的使用一样，不必了解I/O设备的细节。
- ◆块设备文件：表示文件系统高速缓存的设备，例如硬盘驱动器
- ◆设备文件：表示非高速缓存的设备，例如磁带驱动器、鼠标和系统终端

```
[alex@alex/dev]$ls -la tty
```

```
crw-rw-rw- 1 root root  5, 0  Apr 23 19:59 tty
```

管道文件

◆管道文件（named pipe），主要用于在进程间传递数据

```
[alex@alex~]$mknod mypipe p  
[alex@alex~]$ls -l mypipe  
prw-rw-r-- 1 alex alex 0 Apr 24 09:14 mypipe
```

符号链接文件

- ◆ 符号链接文件（symbolic link），这个文件包含了另一个文件的路径名；相当于windows里的快捷式文件

```
[alex@alex~]$ls -l minicom.log  
-rw-rw-r-- 1 alex alex 2903794 Apr 23 15:01 minicom.log  
  
[alex@alex~]$ln -sf minicom.logmlink  
  
[alex@alex~]$ls -lmlink  
lrwxrwxrwx 1 alex alex 4 Apr 23 22:32mlink -> minicom.log
```


文件的全局错误码

- ◆在设计访文件的函数时，会经常出错，所以我们希望能通过一个全局的监听错误信息的机制，C语言提供了一个全局变量`errno`，当文件访问出错的时，全局变量`errno`会记录错误信息码。类似于访问网页出现400代表访问网页不存在的错误码
- ◆`errno`存在于`errno.h`头文件里面，可以利用函数将错误码对应的错误信息取出来

文件的错误信息处理函数

- ◆ `strerror` 取出错误信息

- ◆ 头文件: `#include <string.h>`

- ◆ `char *strerror(int __errnum);`

- ◆ `perror` 打印错误信息

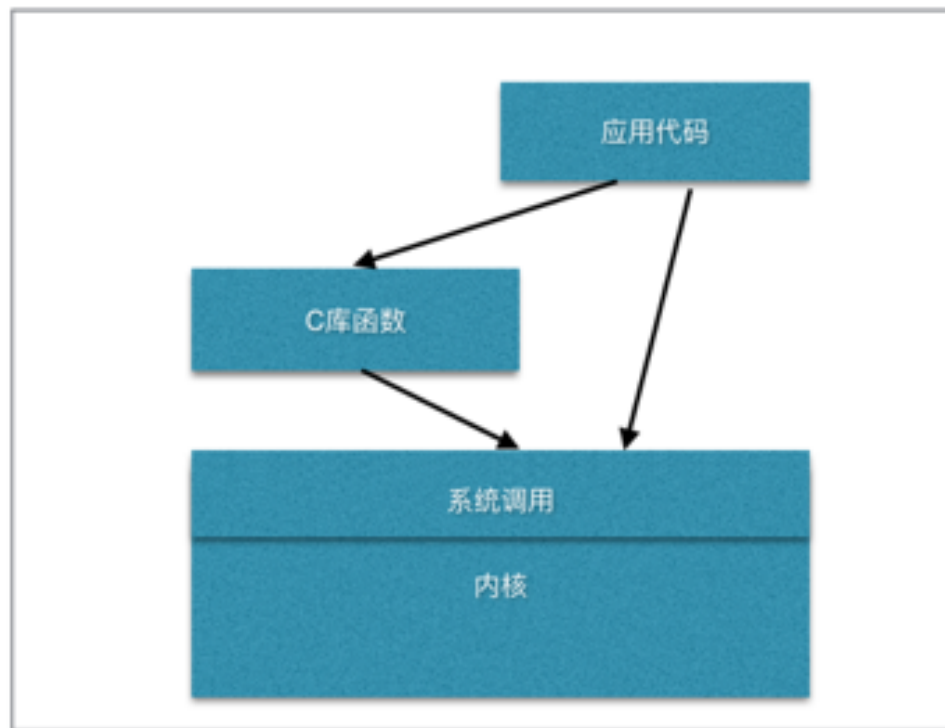
- ◆ 头文件: `#include <stdio.h>`

- ◆ `void perror(const char *);`

操作演示

Linux下对文件的操作方式

- ◆ 文件描述符的操作，属于系统直接调用，处在内核空间
- ◆ 对文件指针的操作，属于借助于C库函数与内核打交道，处在用户空间



文件描述符

◆ 定义：是一个非负整数，是一个文件的索引。

(1) 指的是内核为每一个进程所维护的那张文件表里面的索引值

(2) 当文件被打开，内核会返回一个值，这个值存在打开文件的进程所维护的表里面

(3) 将来该进程就可以通过该索引值对具体的文件进行操作

(4) 从 ≥ 3 开始，系统已经占用了3个描述符

0：标准输入

1：标准输出

2：标准输出错误

文件描述符和文件指针的区别

- ◆ 文件指针：实际上就是一个指向FILE的结构体，FILE结构包括一个缓冲区和一个文件描述符。

stdin:

stdout:

stderr:

- ◆ 文件描述符是文件描述符表的一个索引，因此从某种意义上说文件指针就是句柄的句柄

文件描述符→文件指针

函数原型：FILE *fdopen(int fd, char *opentype);

头文件：#include<stdio.h>

参数：参数一：需要打开的文件描述符

参数二：打开文件的方式,与fopen函数的参数一样

返回值：成功返回打开的文件指针，失败为NULL

文件指针→文件描述符

函数原型：int fileno(FILE *fp);

参数：参数一：需要转换的文件指针

返回值：成功返回打开的文件描述符，失败<0

系统普通文件的API

◆ 常用的文件函数

- ◆ `creat()` 创建文件
- ◆ `open()` 打开/创建文件
- ◆ `write()` 写数据
- ◆ `read()` 读数据
- ◆ `lseek()` 设置偏移量
- ◆ `access()` 判断文件的权限

◆ 头文件

- ◆ `#include <fcntl.h>`
- ◆ `#include <unistd.h>`
- ◆ `#include <sys/types.h>`
- ◆ `#include <sys/stat.h>`

creat 创建文件

函数原型: `int creat(const char *, mode_t)`

参数: 参数一: 文件路径

参数二: 文件创建的权限

文件拥有者的权限: `S_IRWXU(rwx)` `S_IRUSR(r)`
`S_IWUSR(w)` `S_IXUSR(x)`

组的权限: `S_IRWXG` `S_IRGRP` `S_IWGRP` `S_IXGRP`

其他用户的权限: `S_IRWXO` `S_IROTH` `S_IWOTH` `S_IXOTH`

返回值: 成功返回创建好文件描述符, 失败<0

注意点: 无论文件存不存在, 都会重新创建新的文件覆盖原来的文件

open 打开文件

函数原型:

```
int open(const char *pathname, int flags);
```

```
int open(const char *pathname, int flags, mode_t mode);
```

参数:

参数一: 文件的路径

参数二: 文件的打开方式

参数三: 在文件创建的时候使用, 指定创建的方式

返回值: 成功返回打开文件的描述符, 失败<0

文件打开方式

O_RDONLY	以只读方式打开文件
O_WRONLY	以只写方式打开文件
O_RDWR	以读写方式打开文件
O_APPEND	以追加模式打开文件，在每次写入操作指向之前，自动将文件指针定位到文件末尾。但在网络文件系统进行操作时却没有保证。
O_CREAT	如果指定的文件不存在，则按照 mode 参数指定的文件权限来创建文件。
O_TRUNC	文件存在，并且以只读或者只写的方式打开，清空文件
O_CREAT O_EXCL	如果文件已经存在， open 将返回失败。

write 写入数据

函数原型: `ssize_t write(int fd, const void *buf, size_t count);`

参数: 参数一: 文件描述符

参数二: 要往文件里写的数据的地址

参数三: 要往文件里写多少数据, 字节为单位

返回值: 成功返回正确写入文件的字节数, 可能比指定要写入的字节数小; 失败 < 0

write注意点

◆ write出错的原因：

- ◆ 写入的数据超过了一个给定进程文件的长度限制（常见错误）
- ◆ EINTR表示在写的时候出现了中断错误
- ◆ EPIPE表示网络的错误（对方的通信连接关闭）

read读取文件

函数原型: `ssize_t read(int __fd, const void * __buf, size_t __nbyte);`

参数: 参数一:文件描述符

参数二:存放数据的地址

参数三:要从文件里读多少数据, 字节为单位

返回值: 成功返回正确读取文件的字节数,可能比指定的字节数要小;失败<0;当返回值为0时,表示到达了文件末尾,无数据可读

read注意点

```
ssize_t read(int fd, void *buf, size_t len);  
int ret = read(fd, buf, len);
```

- ◆ 1. 此调用的返回值等于len, 所读取的len个字节会被存放到buf中。这是预期的结果
- ◆ 2. 此调用的返回值小于len, 但是大于零。所读取的字节会被存放到buf中。在读取到len个字节之前先到达了文件的末端。
- ◆ 3. 此调用的返回值若为0, 表示EOF(到达了文件末端)。没有数据可读。
- ◆ 4. 此调用的返回值若为-1, 读取失败

lseek重新设置文件描述符的偏移量

函数原型： `off_t lseek(int handle, off_t offset, int fromwhere);`

参数： 参数一： 文件描述符

参数二： 文件的偏移量

参数三： 相对位置（注意：只有当相对位置是
SEEK_END或者是SEEK_CUR，第二个参数才可以是负数）

SEEK_END——>文件尾部

SEEK_SET——>文件头部

SEEK_CUR——>文件的当前位置

返回值： 成功返回文件当前的偏移量，失败<0

access判断文件的相关属性

函数原型：int access(const char *, int);

参数： 参数一:文件的绝对路径

参数二:判断文件的操作权限，有以下参数

返回值：成功返回0，失败<0

R_OK	判断文件是否有读权限。
W_OK	判断文件是否有写权限。
X_OK	判断文件是否有可执行权限。
F_OK	判断文件是否存在。

lstat 获取文件属性

函数原型：
`int lstat(const char *, struct stat *);`
`int stat(const char *, struct stat *);`
`int fstat(int, struct stat *);`

参数：lstat 文件路径，存储文件信息的结构体地址

stat 文件路径，存储文件信息的结构体地址

fstat 文件描述符，存储文件信息的结构体地址

返回值：成功返回0，失败-1

三者的区别：stat 返回文件本身的结构信息。

当一个文件有链接文件时，返回符号链接文件的信息，不是返回文件本身的信息。

fstat 需要打开一个文件，才能访问文件的信息

获取文件类型常用宏

- ◆ S_ISREG () 普通文件
- ◆ S_ISDIR () 目录文件
- ◆ S_ISBLK () 块文件
- ◆ S_ISCHR () 字符设备
- ◆ S_ISLNK () 符号链接文件
- ◆ S_ISFIFO () 管道文件
- ◆ S_ISSOCK () 套接字文件

练习

◆练习1: 实现对普通文件的拷贝

◆练习2: 将一组学生的数据保存到文件，将其读出并输出

目录文件操作的相关函数

◆ 目录文件的操作函数

- ◆ `opendir()`

- ◆ `closedir()`

- ◆ `readdir()`

- ◆ `mkdir()`

- ◆ `rmdir()`

- ◆ `rename()`

◆ 头文件

- ◆ `#include <sys/stat.h>`

- ◆ `#include <unistd.h>`

- ◆ `#include <stdio.h>`

- ◆ `#include <dirent.h>`

opendir打开目录

函数原型：DIR *opendir(const char *);

参数：目录的绝对路径

返回值：成功返回目录流指针，失败为NULL

readdir读取目录里面的内容

函数原型： `struct dirent *readdir(DIR *);`

参数：已打开的文件目录指针

返回值：读不到目录为NULL

closedir 关闭目录流

函数原型：int closedir(DIR *);

参数：目录流的指针

返回值：失败<0

mkdir创建目录

函数原型：int mkdir(const char *, mode_t);

参数： 参数一： 文件路径

参数二： 目录权限，mode值，值的方式参照 chmod的方式0777，注意0不能省略，表示八进制

文件属主对文件的操作权限

文件属主的其他用户对文件的操作权限

其他用户对文件的操作权限

返回值： 成功返回0，失败<0

注意点： 文件的创建一定要有可执行权限； 文件存在，失败

删除普通文件或者是目录remove

函数原型：int remove(const char *);

参数：删除的文件名或者是目录名称

返回值：成功返回0，失败<0

重命名普通文件或者是目录文件

rename

函数原型：int rename (const char *__old, const char *__new);

参数： 参数一：源文件名

参数二：目标文件名

返回值：成功返回0，失败<0

练习

- ◆ 练习1: 实现一个函数删除非空目录
- ◆ 练习2: 目录的拷贝（目录里面有子目录）
- ◆ 练习3: 在不覆盖原来的普通文件，将数据添加到文件指定的位置中